

ECE 116 Final Project

Model gated embedded application

Fall 2026 · Individual work · 25 percent of the course grade

Project overview

ECE116 is about building intelligent embedded systems that turn observations of the physical world into useful behavior. Doing this requires engineering decisions about what to sense, how to interpret uncertain information, and when a response is justified. The value of the system depends on the consequences of those decisions for its intended use.

In this final project, you will bring those decisions together in an application you choose. An embedded device will sense its environment, interpret the input using on-device machine learning, and use that interpretation to gate an action. The local gate makes the system selective about when to act or engage additional processing, so the model's ability to recognize a condition serves a concrete application purpose.

The project culminates A1–A4 by connecting responsive embedded behavior, model evaluation, and deployment under device constraints. Your goal is to demonstrate engineering judgment across that complete sensing-to-action path: use evidence to explain how your choices serve the application and how model errors affect its behavior.

Schedule and submission

You are strongly encouraged to schedule an office-hour meeting with Professor Wang to discuss your project idea. The meeting is optional. Email ejaywang@ucsd.edu to arrange a time.

Due: Friday, December 4, 2026, at 7 p.m. Pacific, through Canvas. Submit the report PDF, a video demonstration lasting 3 to 5 minutes, and the supporting trial data. The report contains four technical pages, one additional page describing AI use, and one additional reference page. The project receives one combined grade. There is no proposal.

Application choices

Choose a useful action that you can build, test, and explain within the project period. Possible directions include:

- Physical actuation or a change in a device's operating state.
- Remote processing whose returned result serves a clear application purpose.
- Mobile interaction or a bounded command to a computer application.
- Event-triggered recording for later inspection.
- Adaptive sensing that enables more detailed capture or analysis when needed.
- Local light, sound, or vibration feedback that guides a user.
- Coordination between a sensing device and another device that performs an action.

You may choose another application that meets the common requirements. A local application using the existing kit can qualify. Added hardware, native phone apps, cloud models, and paid services are optional and receive no extra credit simply for being included.

You may use sensors in the course kit or additional sensors you acquire yourself. The course has no additional budget for project-specific sensors. Select components compatible with the XIAO and follow the course's hardware and data-use rules.

The [optional direction modules](#) provide architecture ideas, prerequisites, design questions, and reference links. They are for your learning and reference; you do not submit separate work for these modules.

What your system must satisfy

1 New on-device classifier

Use a classifier for a task different from the required A2 voice-command and A3–A4 hand-gesture tasks. It must distinguish at least two target classes and an explicit other/background class. Define what belongs in each class and what your other-class examples represent. Explain how the classes influence the application's behavior.

Inference must run on the XIAO ESP32-S3 Sense using real sensor input. A computer or service may support the end-action but cannot replace local inference. Identify the model source/version, input format, preprocessing, output labels, and any training or adaptation. You may reuse earlier acquisition, deployment, and measurement infrastructure. Changing only a threshold on a supplied course model does not satisfy the new-model requirement. You may use a pretrained classifier or train your own; training from scratch is not required.

2 Acquisition and inference timing

Make an explicit design choice about how data is acquired and when the classifier runs. State whether inference is continuous or event-triggered. For continuous operation, report the intended and observed inference interval. For event-triggered operation, define the trigger and the timing or rate limit that governs repeated invocations. State sampling rate and input-window duration where they apply.

Explain how these choices affect responsiveness, missed events, and resource use. Show how the system maintains its required behavior while sensing, classifying, and performing an action.

3 Classifier characterization and deployment

Collect labeled examples from relevant operating conditions. Document collection, class counts, preprocessing, and the separation of development and final evaluation data. Include a confusion matrix and class-level analysis. For each target class, evaluate its score against all remaining classes using a one-versus-rest ROC curve and AUC. Report precision, recall, and false-positive rate at the selected operating point, with counts and denominators.

Measure inference latency on the device. Report model size and memory use, identifying whether each value comes from a physical measurement, compiler report, or profiler estimate. Explain what your evaluation reveals and which conditions remain untested.

4 Explicit event policy and threshold choice

Specify how scores become accepted events. State the threshold or thresholds, class-selection rule, temporal confirmation if used, repeat suppression, and re-arming behavior. Explain the choice if you use no temporal confirmation. Show the policy precisely with a state description, diagram, or concise pseudocode. An other/background prediction must not trigger a target action.

Use measured performance to select your operating point. Explain the effect of false acceptance, missed targets, and confusion between target classes on the end-action. Compare candidate settings on development data, then evaluate the selected configuration without further tuning.

5 Useful action and measured system behavior

An accepted event must cause a meaningful physical or software action. Printing a prediction or displaying its score alone does not satisfy this requirement. Demonstrate intended activation, correct non-activation, and sustained/repeated-input behavior. Report successful actions, false actions, misses, and event-to-action delay with trial counts and timing boundaries.

Analyze model-output failures and potential mitigations as specified in Performance analysis and trial evidence.

Performance analysis and trial evidence

Development and final evaluation

Use development data to compare operating points and choose your classifier configuration and event policy. Keep the conditions comparable when changing a setting. Identify what changes in precision, recall, false positives, action errors, or delay.

Freeze the configuration before collecting the final evaluation. If you revise it after seeing those results, identify the revision and collect a fresh final evaluation. Label development and final records separately.

For the final event-policy evaluation, use at least:

- **20 target trials in total**, covering every target class in your application.
- **20 other/background trials**, when the system should not produce a target action.
- **Two documented operating conditions** across these trials.
- A sustained/repeated-input test and evidence for the model failure analysis.

Report counts separately for each class and condition. These are minimum system-trial counts, not a sufficient dataset size for every classifier analysis. Collect enough labeled examples to support your class-level performance and threshold analysis, and explain the limits of small sample counts. A trial represents a separate event opportunity. Document how opportunities are separated and how the system resets or re-arms.

Evidence supporting the chosen operating point

For each target class, explain what the ROC curve and AUC show about score discrimination. Mark the chosen threshold on the curve and report the precision, recall, and false-positive rate at that threshold. AUC summarizes behavior across thresholds; it does not by itself justify the threshold used in your application.

Connect the numerical results to consequences. Explain which incorrect actions remain possible, which intended actions the system misses, and why you selected this tradeoff over the alternatives you evaluated. Report conflicting evidence and limitations. A general claim that the system is “good enough” does not replace this analysis.

Distinguish classifier performance from event-policy and end-to-end performance. Repeated model windows are not the same as independent application events. If the gate accepts an event but the action fails later, identify that separately from a classification error. Define timing boundaries so inference latency and total response delay are not confused.

Failure analysis

Focus on the consequence of an incorrect model output: false acceptance of other/background, a missed target, or confusion between target classes. Use recorded results or challenging real inputs to examine the failure. Show the model output, the gate’s response, and the resulting end-action behavior. If your tested conditions do not produce an error, report that honestly and clearly distinguish a potential failure from an observed one.

Explain a plausible mitigation for the specific failure. State why it could help, what tradeoff it introduces, and how you would test its effect. You do not need to implement another version. Generic suggestions such as “use more data” need an explanation of what data would address the observed failure. Network outages, storage failures, and security exercises are not required failure demonstrations for this project.

Supporting trial data

Submit the data used to support your plots, metrics, and action results. Include ground-truth labels, class scores, class/condition identifiers, gate decisions, observed actions, and timing where applicable. Preserve ordered records where temporal behavior matters. Document the collection procedure, units, configuration, and which records belong to development or final evaluation. The records must allow a reviewer to trace the report’s claims to real observations.

Report and video

Report format and communication

Submit `Final_Project_<student-id>.pdf` containing **four technical pages**, followed by **one page describing AI use** and **one reference page**. The technical pages include your writing, figures, and tables. Do not add a cover page or further appendices.

Use the supplied `Final_Project_Report_Template.docx` for the two-column technical report. It provides formatting and generic placeholder text only. Choose your own section titles, organization, figures, and sequence of explanation. How clearly you organize and communicate the work is part of the assessment. Remove the template's placeholder content before submission.

The report's purpose is to explain the design decisions and support them with measurements. Cover the required classifier, acquisition/inference timing, deployment, threshold selection, event policy, system results, and failure analysis. Make the implemented configuration identifiable, report the counts behind metrics, and explain the meaning of figures and tables. These are content requirements, not a prescribed outline.

Use the additional AI-use page to identify substantive tools and assistance, their contributions, and what you verified or changed. Use the reference page to credit model/data sources, reused software, and other technical sources. References and AI-use disclosure are outside the four technical pages. Include all six pages in one PDF.

Video demonstration

Submit `Final_Project_<student-id>.mp4`, lasting 3–5 minutes. **The video's purpose is to show the functioning system.** Make the real sensor input, device, gate behavior, and useful end-action observable. Include:

- A brief identification of the application and where inference runs.
- The intended behavior for each target class.
- Other/background input that does not produce a target action.
- A held or repeated input showing repeat suppression and re-arming.
- A challenging or misclassified input and its effect on the action, consistent with your failure analysis. Label any potential rather than observed failure honestly.

The report carries the detailed performance analysis and threshold justification. The video does not need to repeat that analysis or become a narrated slide presentation. Use captions or on-screen text to convey essential narration. Keep cause and effect visible. You may edit the video to combine demonstrations, but it must accurately represent the behavior you observed.

Trial-data file

Submit `Final_Project_<student-id>_Trial_Data.zip` containing the measurement records used in your report and a short description of how you collected and interpreted them. CSV files and recorded device/application traces are suitable. Include the class scores needed to reproduce the classifier metrics and the ordered event records needed to understand the gate. A separate source-code archive or public repository is not required.

Coding support and individual responsibility

You are allowed and encouraged to use coding assistants for generated implementation, libraries, debugging, and refactoring. Explain substantial use on the AI-use page and credit external sources. You remain responsible for understanding the system and testing its behavior.

All submitted performance measurements must come from real work. Do not fabricate or simulate results and present them as device measurements. Do not submit another student's project or use someone else's observations as your own. Follow the course's academic-integrity, hardware, and data-use rules.

Grading and submission

The project is graded out of 100 points and contributes 25 percent of the course grade. The report, video, and supporting data contribute to **one combined project grade**. The video establishes functioning behavior; the report and data establish the analysis supporting the design. The same criteria apply across application directions.

Criterion	Points	Evidence and grading expectations
Classifier deployment and characterization	30	Report and trial data, with device operation visible in the video: at least two target classes plus other; documented model/interface, collection, class-level metrics, ROC/AUC, device latency, and resource use.
Timing and event-policy justification	25	Report and trial data: explicit acquisition/inference scheduling, thresholds, temporal behavior, repeat suppression, and re-arming. Measured operating-point tradeoffs support the choices and their application consequences.
Useful end-action and system evaluation	25	Video, supported by report and trial data: a functioning action controlled by the local classifier/gate; intended, other, and repeated-input behavior; action counts and end-to-end timing.
Failure analysis and proposed mitigation	10	Report and relevant trial/video evidence: trace a model-output failure to its action consequence and explain a specific potential mitigation, expected benefit, tradeoff, and test. No additional implemented version is required.
Communication and attribution	10	Report and video: clear student-chosen organization, readable figures, traceable claims, effective demonstration, and complete AI-use/source attribution.
Total	100	

Within each criterion, complete and supported evidence earns 90–100 percent of its points; substantially correct work with limited gaps earns 70–89 percent; partial implementation with useful diagnostic evidence earns 40–69 percent; minimal relevant evidence earns 1–39 percent; absent evidence earns zero. Apply these bands to the demonstrated portion of each criterion. Missing local inference cannot earn deployment credit, and an absent action cannot earn working-integration credit. Hardware cost, paid services, or visual polish do not replace technical evidence or earn extra credit by themselves.

Canvas submission

Upload the report PDF, demonstration MP4, and trial-data ZIP together to the [final-project Canvas assignment](#). Add all three files before submitting. Retain your working code and original measurements.

After submission, open or download the files to verify the versions, report readability, video playback, and archive contents. If you replace a submission, include all three files again. Staff grade the latest eligible complete submission, subject to approved extensions.

Deadline policy

- **Posted deadline:** Friday, December 4, 2026, at 7 p.m. Pacific.
- **Automatic grace:** through Friday, December 4, at 11:59 p.m., without a deduction or request.
- **Late window:** from Saturday midnight, one 25-percent deduction from the earned score.
- **Hard close:** Monday, December 7, at 7 p.m. Pacific.

A Canvas late label during grace does not imply a deduction. Approved accommodations and emergencies follow the course policy.